

原来如此，是我误解了你的需求！你是想自己写一段代码来生成标准的三次握手、数据传输和四次挥手流量，然后去 Wireshark 里抓取它。这个思路非常棒，实战造轮子是理解底层协议最好的方法。

为了完全符合你的要求，并且严格使用 Python 3.8.6 标准库（仅依赖内置的 `socket` 和 `threading` 模块），我为你编写了下面的脚本。

这个脚本在一个进程内启动了两个线程，一个是服务端，一个是客户端。它们会在本地回环地址（`127.0.0.1`）上建立 TCP 连接、互发问候数据包，然后干净利落地断开。最重要的是，它会在终端直接打印出 Wireshark 的过滤表达式，让你能一键定位。

Python TCP 握手与挥手发包器

```
1  import socket
2  import threading
3  import time
4
5  def tcp_server(host, port, ready_event, info_dict):
6      """TCP 服务端线程"""
7      server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
8      # 允许地址重用，防止快速重启脚本时报错
9      server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
10
11     # 绑定主机和端口（端口传 0 表示让操作系统随机分配一个空闲端口）
12     server_socket.bind((host, port))
13
14     # 获取系统实际分配的服务端端口
15     actual_port = server_socket.getsockname()[1]
16     info_dict['server_port'] = actual_port
17
18     server_socket.listen(1)
19     print(f"[Server] 服务端已启动，正在监听 {host}:{actual_port} ...")
20
21     # 触发事件，通知客户端服务端已经准备好接收连接了
22     ready_event.set()
23
24     # 阻塞等待客户端连接（这里会完成三次握手）
25     conn, addr = server_socket.accept()
26     print(f"[Server] 接受来自客户端的连接: {addr}")
27
28     # 接收客户端发来的数据
29     data = conn.recv(1024)
30     print(f"[Server] 收到客户端数据: {data.decode('utf-8')}")
31
```

```

32     # 给客户端回发数据
33     conn.sendall(b"Hello from Server! I received your message.")
34
35     # 稍微延迟一下，确保数据包完全发送，再发起挥手
36     time.sleep(0.5)
37
38     # 服务端关闭连接（发送 FIN 包，开启四次挥手）
39     conn.close()
40     server_socket.close()
41     print("[Server] 服务端已关闭连接。")
42
43 def tcp_client(host, info_dict, ready_event):
44     """TCP 客户端线程"""
45     # 阻塞等待，直到服务端端口绑定完成并开始 listen
46     ready_event.wait()
47     server_port = info_dict['server_port']
48
49     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
50
51     print(f"[Client] 尝试连接到服务端 {host}:{server_port} ...")
52     # 发起连接（发送 SYN 包，开启三次握手）
53     client_socket.connect((host, server_port))
54
55     # 获取系统为客户端随机分配的本地出站端口
56     client_local_port = client_socket.getsockname()[1]
57     info_dict['client_port'] = client_local_port
58     print(f"[Client] 连接成功！客户端本地分配端口为：{client_local_port}")
59
60     # 发送数据
61     client_socket.sendall(b"Hello from Client! This is a test packet.")
62
63     # 接收服务端回发的数据
64     data = client_socket.recv(1024)
65     print(f"[Client] 收到服务端数据：{data.decode('utf-8')}")
66
67     # 稍微延迟，等待服务端先发起 FIN，然后客户端响应 FIN
68     time.sleep(1)
69     client_socket.close()
70     print("[Client] 客户端已关闭连接。")
71
72 def main():
73     host = '127.0.0.1'
74     port = 0
75

```

```

76     # 用于在线程间共享端口信息的字典
77     info_dict = {}
78     # 用于同步服务端和客户端启动顺序的事件锁
79     ready_event = threading.Event()
80
81     threadRunEnableCard = True
82
83     if threadRunEnableCard:
84         # 创建并启动服务端和客户端线程
85         server_thread = threading.Thread(target=tcp_server, args=(host, port,
ready_event, info_dict))
86         client_thread = threading.Thread(target=tcp_client, args=(host,
info_dict, ready_event))
87
88         server_thread.start()
89         client_thread.start()
90
91         # 等待两个线程执行完毕
92         server_thread.join()
93         client_thread.join()
94
95         print("\n" + "="*60)
96         print("✅ 交互完成! 请将下面这行表达式复制到 Wireshark 的过滤器中: ")
97         print(f"tcp.port = {info_dict['server_port']} and tcp.port =
{info_dict['client_port']}")
98         print("="*60)
99
100 if __name__ == "__main__":
101     main()

```

🎯 配合 Wireshark 食用的 2 个避坑点:

1. **抓错网卡问题（最常见错误）**: 因为代码中的 IP 是 `127.0.0.1`，数据包是不会经过你的物理网卡（如以太网或 Wi-Fi）的。在 Wireshark 启动界面，你**必须**选择监听 `Adapter for loopback traffic capture (Npcap Loopback Adapter)` 这张虚拟网卡，否则什么都抓不到。
2. **操作顺序**: 请**先**在 Wireshark 中选中 Loopback 网卡开始抓包，**然后**再运行这个 Python 脚本。运行结束后，把终端打印出来的那行 `tcp.port = xxx and tcp.port = xxx` 直接复制到 Wireshark 顶部的过滤栏回车，你就能看到这条流独享的、非常干净的握手、传数据、挥手全过程了。